Práctica 3 ADSOF

Alejandro Seguido y Rubén García.

Apartado 1.

Para la representación de las conexiones unidireccionales entre usuarios se ha implementado la clase Enlace, encapsulando la información del usuario origen, el destino y el coste de propagación mediante atributos privados.

En la instanciación de los objetos se ha implementado una lógica de control de errores dentro del constructor principal para garantizar que el coste nunca sea menor o igual a cero, asignando un valor por defecto de 1 en esos casos. Para facilitar la creación de enlaces sin coste explícito, se ha hecho uso de la sobrecarga de constructores aplicando encadenamiento (this(...)), lo que permite reutilizar la lógica de validación del constructor principal.

Para cumplir con el requisito de acumular el coste total de la red, se ha definido una variable de clase (static int costeTotal) que actúa como un registro global compartido por todas las instancias. Esta variable se actualiza automáticamente cada vez que se crea un nuevo enlace a través del constructor. Además, se ha diseñado el método cambiarDestino(Usuario, int) para permitir la modificación de enlaces existentes protegiendo el atributo origen. Este método incluye la lógica necesaria para recalcular y ajustar el contador estático costeTotal, restando el coste antiguo y sumando el nuevo, garantizando así la consistencia del registro global en todo momento.

Apartado 2. Clase básica Usuario

Para almacenar los enlaces salientes de cada usuario se ha utilizado una colección *List<Enlace>* implementada mediante *ArrayList*, lo que permite mantener el orden de inserción de los enlaces y recorrerlos o acceder a ellos mediante índice.

En la construcción de los objetos *Usuario* se han definido varios constructores y se ha aplicado encadenamiento de constructores *(this(...))* para evitar duplicación de código y centralizar la inicialización de atributos.

Para la gestión de enlaces se han implementado dos métodos *addEnlace*. El método que recibe el usuario destino crea el objeto *Enlace* y realiza una llamada al método *addEnlace(Enlace e)*, que realiza todas las comprobaciones necesarias y añade el enlace. Esto permite reutilizar código y mantenerlo extensible.

Problemas abordados

Uno de los principales problemas abordados ha sido evitar la creación de enlaces duplicados hacia un mismo usuario destino, lo cual se ha resuelto mediante un método que comprueba previamente si ya existe un enlace hacia ese usuario.

* Como posibles mejoras, se podrían añadir más comprobaciones de validación en algunos métodos.

Apartado 3.

Para la representación de la información que fluye por la red se ha implementado la clase Mensaje, la cual encapsula su contenido en forma de texto, su alcance disponible y el Usuario en el que se encuentra en cada momento. Su estado interno es dinámico y cambia conforme el mensaje se propaga por los enlaces del sistema.

El método principal de la propagación es mediante el método difunde(Enlace e). Este método actúa como un gestor de transacciones: valida que la difusión sea legal y actualiza el estado del mensaje pasándose al usuario destino, descontando el coste de propagación del enlace y sumando el valor de amplificación otorgado por el nuevo usuario. Además, se ha añadido un control de seguridad para garantizar que el alcance residual nunca tome valores negativos.

Para la comprobación de la propagación se ha implementado dos métodos auxiliares: puedeDifundirPor(Enlace) y aceptadoPor(Usuario). Como estos métodos los utilizan las clases de esta jerarquía, se ha decidido que sean protected. Esto permite que, para un mensaje estándar, la validación sea sencilla (comprobar el coste y devolver siempre true en la aceptación), pero deja la puerta abierta al polimorfismo para que clases derivadas, como el futuro MensajeControlado, puedan sobrescribir y aplicar reglas mucho más restrictivas sin necesidad de alterar el método principal de difusión.

Finalmente, se ha desarrollado una sobrecarga del método difunde para procesar rutas completas haciendo uso de argumentos variables (mediante la sintaxis Usuario... ruta). Se evalúa cada salto de la ruta de forma individual. Si un salto falla (ya sea por inexistencia del enlace, alcance insuficiente o rechazo del destino), el proceso no se interrumpe prematuramente, sino que el bucle continúa intentando contactar con los siguientes usuarios de la lista desde su posición actual. El método consolida el resultado global devolviendo true únicamente si todos y cada uno de los saltos previstos se ejecutaron con éxito.

Apartado 4. Pruebas de las clases anteriores

Se ha desarrollado una clase de pruebas para verificar el funcionamiento de las clases principales del sistema. Las pruebas se organizan en tres bloques, cada uno correspondiente a una clase distinta.

1. Pruebas clase Enlace.

Se comprueba la creación de enlaces con distintos valores de coste (ninguno, positivo, cero y negativo) y el funcionamiento de los métodos getter.
Además, se prueba el cálculo y actualización del coste acumulado, al crear o modificar un enlace, el coste especial y coste real.
Se representan textualmente mediante toString() los enlaces y se evalúa la modificación del destino y del coste del enlace mediante *cambiarDestino*, incluyendo casos en los que el coste introducido es menor o igual a cero.

2. Pruebas clase Usuario.

Se prueba la creación de usuarios mediante distintos constructores y la obtención de atributos mediante getters.
También se prueba la inserción de enlaces usando los dos métodos *addEnlace* disponibles. Obteniendo inserciones válidas y comprobando el retorno de errores al intentar añadir enlaces duplicados, crear enlaces hacia el propio usuario, añadir enlaces con origen incorrecto o nulos.
Además, se valida la obtención de enlaces por el usuario destino o mediante índice, el funcionamiento de *getNumEnlaces* y la representación textual del usuario mediante toString() con distintos números de enlaces.

3. Pruebas clase Mensaje.

Se verifica la creación de mensajes, el correcto funcionamiento de getters y setters, y la representación textual mediante toString().
Se evalúa la difusión del mensaje tanto a través de un enlace como mediante una ruta de usuarios. También se comprueban las condiciones de error, incluyendo ausencia de enlace entre usuarios, alcance insuficiente, enlaces nulos, listas de usuarios nulas y rutas de difusión vacías.

Apartado 5.

Para procesar los ficheros, extraer su información, crear y almacenar los objetos descritos se han implementado tres funciones, cada una dedicada al procesamiento de un fichero distinto. Se ha extraído la correspondiente información leyendo el fichero por líneas, separando las palabras por espacios usando el método *split("\\s+")*, ideal porque trata cualquier cantidad de espacios en blanco como un único separador. Además, se utilizan métodos propios de la *RedSocial* para crear y añadir tanto los usuarios, enlaces y el mensaje.
El almacenamiento de los distintos objetos se ha realizado con la ayuda de un mapa de usuarios con clave su nombre (que nos permite localizar usuarios de forma casi instantánea, resolviendo el problema de la vinculación de datos de manera eficiente), un puntero al mensaje inicial y una lista de usuarios, que corresponde con los usuarios que el mensaje intentará visitar.

Una vez cargada la red, el sistema debe "difundir" el mensaje siguiendo una ruta predefinida. Este método invoca al método *difunde(Enlace e)* de la clase *Mensaje*, y en caso de ser exitosa la difusión, se imprime el mensaje por la terminal.

En cuanto a la gestión de excepciones, hemos delegado la responsabilidad del error hacia arriba. Tanto los métodos de lectura como escritura en fichero, lanzan excepciones que tendrán que ser gestionadas por el programa que utilice esta clase mostrando el mensaje adecuado.

Extensión de la clase *RedSocial* en una *Fachada*.
Se han creado las métodos adecuadas para crear usuarios (*addUsuario*), enlaces (*addEnlace*), o añadir un mensaje (*addMensaje*), que sobrescribirá sobre el mensaje previo, antes almacenado. Además, se han añadido métodos para añadir usuarios a la ruta de destinos del mensaje y para vaciar la ruta. Estos métodos permiten manejar y usar las clases *Usuario*, *Enlace* y *Mensaje* de una forma mucho más simple y abstracta, sin centrarte en los detalles técnicos correspondientes de cada clase, haciendo una fachada funcional y fácil de usar.

Finalmente, para transformar la clase en una Fachada, implementamos métodos como *addUsuario* o *addEnlace* que encapsulan la creación de objetos. Esto resuelve el problema de la complejidad: quien use la *RedSocial* no necesita saber cómo se instancia un *Enlace* o cómo se vincula a un *Usuario*; simplemente "pide" que se cree. Además, añadimos la funcionalidad de guardado, asegurando que el formato de salida sea idéntico al de entrada, permitiendo que la red pueda ser guardada y recuperada en sesiones futuras sin pérdida de integridad.

Prueba uso Red Social.

Se ha extendido la prueba de uso de la clase *RedSocial*, añadiendo los métodos implementados para convertir la clase a fachada.
En este nuevo caso de uso, se añaden nuevos usuarios y enlaces, y un nuevo mensaje que será difundido por una ruta de difusión distinta creada desde cero. Además, se guardará el

estado de la red social antes de difundir el mensaje, para comprobar la correcta escritura en ficheros y la correcta inserción de los nuevos objetos añadidos a la red social.

Apartado 6.

Para el apartado 6 se ha ido completando por bloques, extendiendo cada clase con su correspondiente mejora.

En la primera parte se pide crear un nuevo tipo de Enlace que sea un señuelo. Para ello se decidió crear una clase llamada EnlaceSenuelo que se extienda de la clase Enlace, esta nueva clase hereda los métodos de la clase padre y se construye con nuevos parámetros como son probabilidad de retorno y factor de coste extra, pero se llama al constructor del padre con la función super. Para las nuevas funcionalidades se hace un override de los métodos del padre redefiniendo cómo van a funcionar, sobreescribimos el método de coste especial y el de get UsuarioDestino, cambiando la lógica interna.

Para ampliar el comportamiento de los usuarios, se introdujo la clase UsuarioInteresado que hereda directamente de Usuario. El objetivo de esta clase es alterar la lógica de enrutamiento de los mensajes para priorizar aquellos destinos con mayor visibilidad, preservando al mismo tiempo el orden de creación de los enlaces almacenados en la clase padre. Para lograrlo, se ha sobrescrito el método getEnlace. La nueva implementación recorre secuencialmente la lista de enlaces y evalúa la exposición del usuario destino; si esta es ALTA o VIRAL, devuelve inmediatamente dicho enlace, ignorando el destino original solicitado. En caso de no encontrar ningún contacto con alta exposición, actúa como mecanismo de respaldo y recurre al comportamiento estándar invocando a super.getEnlace.

Finalmente, se implementó la clase MensajeControlado, la cual extiende la funcionalidad de la clase base Mensaje incorporando un nuevo atributo numérico denominado rigidez. Para cumplir con las estrictas políticas de difusión que exige este tipo de mensaje, se ha hecho uso del polimorfismo sobrescribiendo dos métodos protegidos clave: puedeDifundirPor y aceptadoPor. En el método puedeDifundirPor, se restringe de forma absoluta el paso por enlaces engañosos comprobando si el enlace proporcionado es una instancia de EnlaceSenuelo (mediante el uso del operador instanceof); si no lo es, delega la comprobación de alcance a la clase padre. Por otro lado, en el método aceptadoPor se evalúa si el usuario destino es apto para recibir el mensaje basándose en su nivel de exposición. Esto se ha resuelto mediante una estructura switch que implementa los umbrales mínimos de rigidez definidos en los requisitos (por ejemplo, exigiendo una rigidez mínima de 20 para usuarios con exposición ALTA).
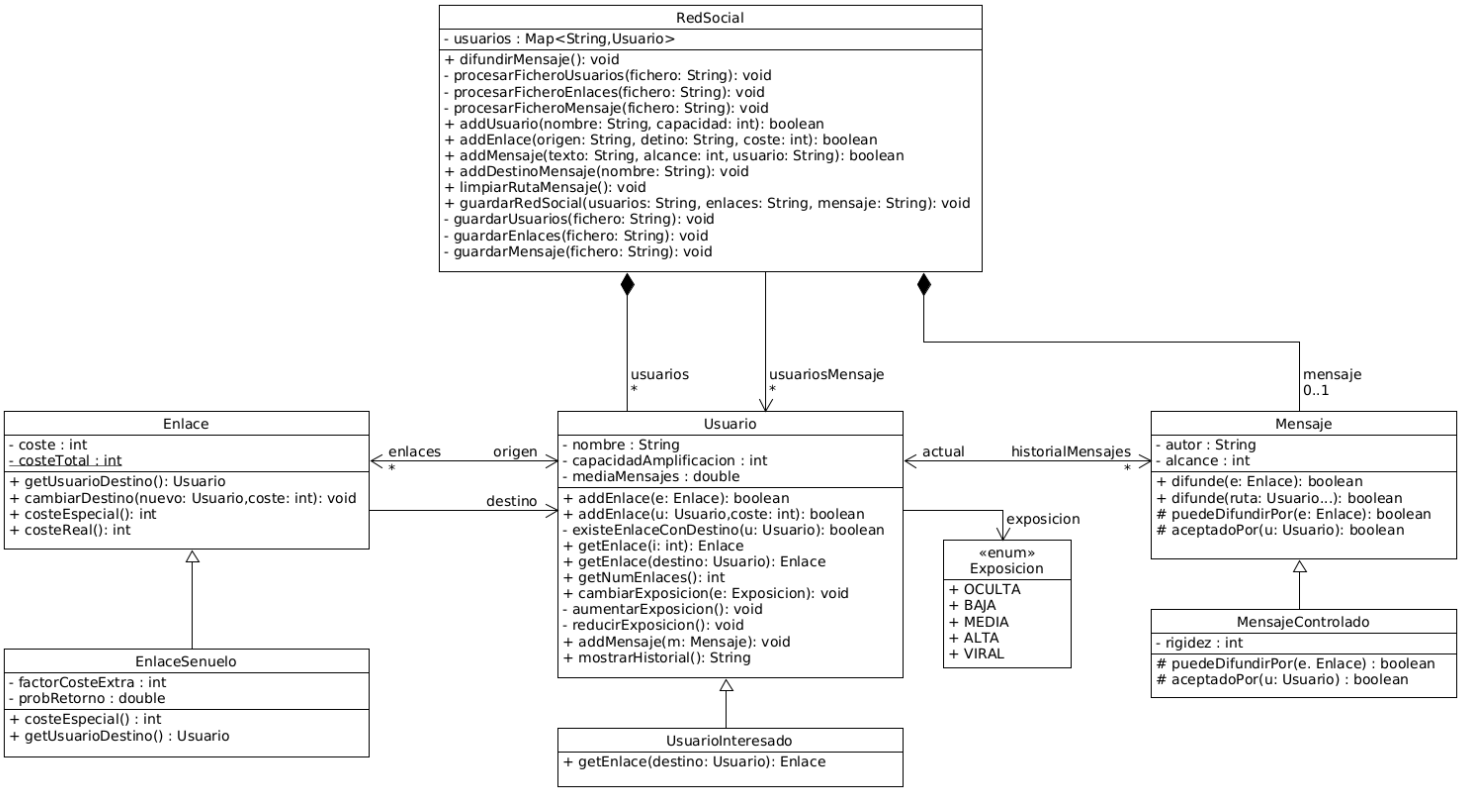
Diagrama de clases.